



Génie Logiciel – L3 Informatique

31 mars 2026

Rapport du projet : IA d'Échecs

FAVRIOU Valentin
ARANEGA Noah

Encadrant: ANTUNES Benjamin

Sommaire :

Introduction	2
Présentation de l'Applicaton	4
Présentation de l'architecture logicielle	5
Diagramme de classes UML	6
Diagramme de séquence UML	7
Patrons de Conception utilisés	8
Outils logiciels utilisés	9
Langage et compilation	9
Bibliothèque graphique	9
Environnements de développement	9
Débogage et analyse mémoire	9
Gestion de versions	10
Parallélisme	10
Tableau des Tâches	11
Difficultés rencontrées	12
Conclusion	13

Introduction

Le jeu d'échecs est un des jeux les plus anciens de l'histoire qui est encore populaire de nos jours. Faisant son apparition autour du 6ème siècle après J-C, il mélange stratégie, anticipation, prise de décisions dans un environnement complexe et réflexion du joueur.

Le jeu d'échecs oppose deux armées, une blanche et une noire, composées de 8 pions, 2 fous, 2 cavaliers, 2 tours, 1 reine et 1 roi sur un plateau de 8 cases par 8 cases. Le but est de mettre en échec et mat le joueur adverse, c'est-à-dire de faire en sorte que le roi soit attaqué sans que le joueur puisse le bouger ou le défendre.

Chaque pièce a des déplacements spéciaux:

- Le **pion** peut se déplacer en avant d'une case sauf si la case est bloquée par une autre pièce. Son premier coup peut avancer de deux cases, et il ne peut manger une pièce que si elle se trouve en diagonale une case devant elle.
- Le **fou** peut se déplacer qu'en diagonale, il ne peut pas passer à travers une pièce mais peut la manger si elle se trouve dans ses diagonales.
- Le **cavalier** peut se déplacer en forme de L, il peut passer au dessus de n'importe quelle pièce et peut la manger si elle se trouve sur sa case de déplacement. (ex: si le cavalier se trouve en (5,5), il peut aller en (3,6), (3,4), (7,6), (7,4), (6,3), (4,3), (6,7) et (4,7).)
- La **tour** peut se déplacer verticalement et horizontalement (dans sa colonne et sa ligne), tout comme le fou il ne peut pas passer au dessus d'une pièce mais il peut la manger si elle est sur son chemin.
- La **reine** hérite des déplacements de la tour et du fou, elle peut se déplacer en diagonales et sur sa ligne et sa colonne.
- Le **roi** peut bouger d'une case dans toutes les directions (nord, est, sud, ouest, nord-est, sud-est, sud-ouest et nord-ouest), mais il doit faire attention à ne pas être en échec s'il se déplace sur une de ces cases.

Le jeu d'échecs est un jeu très complexe qui requiert beaucoup de capacités intellectuelles pour pouvoir anticiper plusieurs coups d'avances et par conséquent être fort à ce jeu. L'Intelligence Artificielle (IA) a réussi à impacter ce jeu, notamment en 1997 lorsque le superordinateur IA Deep Blue bat le champion du monde Garry Kasparov.

Cet épisode de l'histoire des échecs nous a inspiré à recréer une IA qui soit capable de jouer aux échecs de manière autonome, en analysant la meilleure décision à prendre dans chaque position où elle se trouve.

Ce rapport présentera tout d'abord l'application que l'on a conçue, pour ensuite

présenter l'architecture logicielle, suivi du diagramme des classes UML de notre projet, nous rebondirons ensuite sur les patrons de conceptions que nous avons utilisé pour notre projet, puis nous aborderons ensuite les outils logiciels utilisés, les tâches effectuées et les difficultés rencontrées.

Présentation de l'Applicaton

Présentation de l'architecture logicielle

L'application a été conçue selon une architecture modulaire, permettant de séparer clairement les différentes responsabilités du système. Cette organisation facilite la compréhension, la maintenance et l'évolution du code.

L'architecture repose sur plusieurs composants principaux :

Tout d'abord, la gestion du jeu est assurée par un ensemble de classes représentant les éléments fondamentaux des échecs. La classe `Piece` constitue une classe de base dont héritent les différentes pièces du jeu telles que `Pawn`, `Rook`, `Knight`, `Bishop`, `Queen` et `King`. L'échiquier et les positions sont gérés à l'aide des classes `Player` et `Coordinates`, permettant de représenter l'état du jeu à tout instant.

La logique du jeu est centralisée dans la classe `GameController`, qui gère le déroulement d'une partie, les déplacements des pièces, ainsi que les règles du jeu (déplacements valides, échecs, captures, etc.). Cette classe joue un rôle central dans la coordination des différents composants du projet.

Le module d'intelligence artificielle est principalement composé des classes `Minimax` et `Evaluator`. La classe `Minimax` implémente un algorithme de recherche permettant d'explorer les coups possibles afin de déterminer le meilleur choix. Elle s'appuie sur la classe `Evaluator`, qui attribue un score à une position donnée en fonction de différents critères (valeur des pièces, position sur l'échiquier, etc.). Cette séparation permet de distinguer clairement la logique de recherche de celle d'évaluation.

L'interface utilisateur est divisée en deux parties : une interface graphique (`GraphicDisplay`) et une interface terminale (`TerminalDisplay`). Ces composants permettent d'interagir avec le joueur et d'afficher l'état du jeu. Des classes supplémentaires telles que `Button`, `Shape` ou `TextShape` sont utilisées pour gérer les éléments graphiques.

Enfin, des classes utilitaires comme `MoveHistory` permettent de suivre les coups joués, tandis que `Player` représente les joueurs et leurs pièces.

Les différents modules interagissent de manière structurée. Par exemple, l'intelligence artificielle utilise l'état du jeu fourni par `GameController` pour simuler des coups, tandis que l'interface récupère ces informations pour les afficher à l'utilisateur.

Cette architecture repose sur une séparation claire des responsabilités, rendant le code plus modulaire, maintenable et évolutif.

Diagramme de classes UML

Diagramme de séquence UML

Patrons de Conception utilisés

Modèle-Vue-Contrôleur :

Modèle Vue Contrôleur

Memento : Historique des coups

Outils logiciels utilisés

Langage et compilation

Ce projet a été développé en **C++20**, un langage de programmation compilé offrant des performances élevées, adaptées aux algorithmes de recherche intensifs comme le Minimax. La compilation est gérée par un **Makefile** utilisant **g++** avec les flags d'optimisation `-O2` pour les builds de production et `-g` pour le débogage.

Bibliothèque graphique

L'interface graphique a été réalisée à l'aide de **Raylib**, une bibliothèque C légère et multiplateforme orientée jeu vidéo. Elle nous a permis de gérer l'affichage du plateau, des pièces, et les interactions souris

Environnements de développement

Deux environnements ont été utilisés selon les besoins :

- **Docker** : un conteneur basé sur Ubuntu 22.04 garantit un environnement de compilation reproductible et isolé, facilitant le déploiement et l'intégration continue. Il embarque toutes les dépendances nécessaires (Raylib, g++, CMake, Valgrind).
- **Guix** : un gestionnaire de paquets fonctionnel utilisé pour le développement local. Contrairement à Docker, Guix s'exécute directement sur la machine hôte, offrant un accès natif à l'affichage X11 sans configuration supplémentaire. La commande `guix shell --manifest=guix.scm` instancie un environnement isolé avec exactement les paquets déclarés.

Débogage et analyse mémoire

Valgrind a été utilisé pour détecter les fuites mémoire et les accès invalides, notamment sur les structures de données complexes comme les `shared_ptr` et les vecteurs de pièces.

Gestion de versions

Le projet a été versionné avec **Git** et hébergé sur **GitHub**, permettant une collaboration efficace entre les membres de l'équipe et un suivi précis de l'historique des modifications.

Parallélisme

La recherche du meilleur coup par l'IA exploite le parallélisme système via les appels **POSIX** `fork()` et `pipe()`. Chaque coup candidat à la racine de l'arbre Minimax est évalué dans un processus fils indépendant, les résultats étant renvoyés au processus père via des tubes de communication.

Tableau des Tâches

Difficultés rencontrées

Conclusion